

Week 5 Assignment

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Ans:-

- Stores intermediate results on disk, causing high I/O overhead.
- Slow for iterative and real-time processing.
- Complex programming model.
- Not suitable for interactive analytics.
- Higher execution time compared to Spark.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Ans:-

Spark stores intermediate data in memory instead of writing it to disk after every operation. This significantly speeds up iterative machine learning algorithms because the same data can be reused multiple times without repeated disk reads and writes.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

Ans:-

```
df = df.dropDuplicates(["user_id", "transaction_date"])
```

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

Ans:-

```
from pyspark.sql.functions import avg

df_sales.filter(df_sales.region == "West") \
    .groupBy("product_category") \
    .agg(avg("sale_amount").alias("avg_sale_amount")) \
```

`.show()`

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

Ans:-

`.na.drop()` removes rows containing null values.

`.na.fill()` replaces null values with a specified value.

Example:

```
df = df.na.fill({"status": "Unknown"})
```

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Ans:-

```
from pyspark.sql.functions import count
```

```
df.groupBy("city") \
```

```
  .agg(count("*").alias("total_records")) \
```

```
  .filter("total_records > 100") \
```

```
  .show()
```

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Ans:-

Spark DataFrames are immutable, meaning they cannot be modified directly. Operations such as dropping or renaming columns create a new DataFrame instead of changing the original one.

Example:

```
new_df = df.drop("age")
```

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Ans:-

```
from pyspark.sql.functions import col

df.filter(

    (col("age") >= 18) &

    (col("age") <= 30) &

    (col("subscription") == "Premium")

).show()
```

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Ans:-

Null values can affect calculations and may lead to inaccurate or incomplete results. Handling null values before applying functions such as sum() and avg() ensures more reliable analysis.

Q10: Write the code to **revise** a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

Ans:-

```
from pyspark.sql.functions import col

from pyspark.sql.types import TimestampType

df = df.withColumn(

    "event_time",

    col("raw_timestamp").cast(TimestampType())

).drop("raw_timestamp")
```

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Ans:-

A shuffle occurs when Spark redistributes data across partitions during operations such as `groupBy()` and `join()`. It is considered a wide transformation because data from multiple partitions must be exchanged before processing.

Q12: Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

Ans:-

```
from pyspark.sql.functions import col, trim

df = df.filter(

    col("email").isNotNull() &

    (trim(col("username")) != "")

)
```

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Ans:-

```
from pyspark.sql.functions import min, max, avg

df.agg(

    min("price").alias("min_price"),

    max("price").alias("max_price"),

    avg("price").alias("mean_price")

).show()
```

Q14: In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Ans:-

If date formats are inconsistent, Spark may infer incorrect data types or treat dates as strings. This can lead to parsing errors, incorrect calculations, and unreliable analysis.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by store_id to calculate total revenue.

Ans:-

```
from pyspark.sql.functions import sum
result = (
    df.dropDuplicates()
    .na.fill({"price": 0})
    .groupBy("store_id")
    .agg(sum("price").alias("total_revenue"))
)
result.show()
```